Name: Vanshika Wadhwani
Class: D20A
Roll no:35

Experiment:2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.

2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).

3. **Hashing** – SHA-256 algorithm ensures data immutability.

4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.

5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

Step 1: Install required libraries

```
!pip install flask==2.2.5 pyngrok

import datetime

import hashlib

import json
```

from flask import Flask, jsonify

Step 2: Create the Blockchain class

```
    def __init__(self):
        self.chain = []
        self.transactions = []

        # Create Genesis Block
        self.create_block(
            proof=1,
            previous_hash='0'
        )

    # Create a new block
    def create_block(self, proof, previous_hash):

        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'transactions': self.transactions.copy(),
            'previous_hash': previous_hash
        }

        self.chain.append(block)

        return block

    # Get the previous block
    def get_previous_block(self):
        return self.chain[-1]

    # Add a transaction
    def create_transaction(self, sender, receiver, amount):

        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }
```

```
    self.transactions.append(transaction)

    return self.get_previous_block()['index'] + 1

# Proof of Work
def proof_of_work(self, previous_proof):

    new_proof = 1

    while True:

        hash_operation = hashlib.sha256(
            str(
                new_proof ** 2 - previous_proof ** 2
            ).encode()
        ).hexdigest()

        # Hash must start with 000
        if hash_operation[:3] == '000':
            return new_proof

        new_proof += 1

# Generate SHA-256 hash of a block
def hash(self, block):

    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()

    return hashlib.sha256(
        encoded_block
    ).hexdigest()

# Validate blockchain
def is_chain_valid(self, chain):

    previous_block = chain[0]
    block_index = 1

    while block_index < len(chain):

        block = chain[block_index]
```

```
            # Check previous hash
            if block['previous_hash'] != self.hash(previous_block):
                return False

            previous_proof = previous_block['proof']
            proof = block['proof']

            # Verify Proof of Work
            hash_operation = hashlib.sha256(
                str(
                    proof ** 2 - previous_proof ** 2
                ).encode()
            ).hexdigest()

            if hash_operation[:3] != '000':
                return False

            previous_block = block
            block_index += 1

        return True
```

Step 3: Create blockchain and add transaction

```
blockchain = Blockchain()

block_index = blockchain.create_transaction(
    sender="Vanshika Wadhwani",
    receiver="VESIT",
    amount=100
)

print("Transaction added successfully!")
print("Transaction will be included in Block:", block_index)

print("\nPending Transactions:")
print(blockchain.transactions)
```

```
Transaction added successfully!
Transaction will be included in Block: 2

Pending Transactions:
[{'sender': 'Vanshika Wadhwani', 'receiver': 'VESIT', 'amount': 100}]
```

Step 4: Mine the block

```
previous_block = blockchain.get_previous_block()

previous_proof = previous_block['proof']

print("Mining block...")

proof = blockchain.proof_of_work(previous_proof)

print("Proof found:", proof)

previous_hash = blockchain.hash(previous_block)

new_block = blockchain.create_block(
    proof,
    previous_hash
)

# Clear pending transactions
blockchain.transactions.clear()

print("\nBlock mined successfully!")
print(new_block)
```

```
Mining block...
Proof found: 533

Block mined successfully!
{'index': 2, 'timestamp': '2026-09-07 08:51:19.570885', 'proof': 533, 'transactions': [{'sender': 'Vanshika Wadhwani', 'receiver': 'VESIT', 'amount': 100}], 'previous_hash': '4187ae1432f29353f58d5
```

Step 5: Display and validate blockchain

```
print("================================")
print("COMPLETE BLOCKCHAIN")
print("================================")
```

```
print(json.dumps(blockchain.chain, indent=4))

print("\n================================")
print("BLOCKCHAIN VALIDITY")
print("================================")

if blockchain.is_chain_valid(blockchain.chain):
    print("All good. The Blockchain is valid.")
else:
    print("The Blockchain is not valid.")
```

```
================================
COMPLETE BLOCKCHAIN
================================
[
    {
        "index": 1,
        "timestamp": "2026-09-07 08:50:52.251513",
        "proof": 1,
        "transactions": [],
        "previous_hash": "0"
    },
    {
        "index": 2,
        "timestamp": "2026-09-07 08:51:19.570885",
        "proof": 533,
        "transactions": [
            {
                "sender": "Vanshika Wadhwani",
                "receiver": "VESIT",
                "amount": 100
            }
        ],
        "previous_hash": "4187ae1432f29353f58d5c15f9b92b6c66f6682701860cdf9030e699ee682ad0"
    }
]

================================
BLOCKCHAIN VALIDITY
================================
All good. The Blockchain is valid.
```

Step 6: Implement Flask APIs

```
@app.route('/mine_block', methods=['GET'])
def mine_block():

    if len(blockchain.transactions) == 0:
        return jsonify({
            'message': 'No transactions available.'
        }), 400

    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)
```

```
    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(
        proof,
        previous_hash
    )

    blockchain.transactions.clear()

    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'transactions': block['transactions'],
        'previous_hash': block['previous_hash']
    }

    return jsonify(response), 200
```

Step 7: Run Flask and Test Using Postman

```
!pip install pyngrok
```

Configure ngrok:

```
from pyngrok import ngrok

ngrok.set_auth_token("YOUR_NGROK_AUTH_TOKEN")
```

Run Flask:

```
import threading


def run_flask():


    app.run(
        host='0.0.0.0',
        port=5000,
        debug=Fals
        e,
```

```
    use_reloader=False
  )

thread =
threading.Thread(target=run_flask)
thread.start()
```

Create the public URL:

```
import time
from pyngrok import ngrok

time.sleep(3)

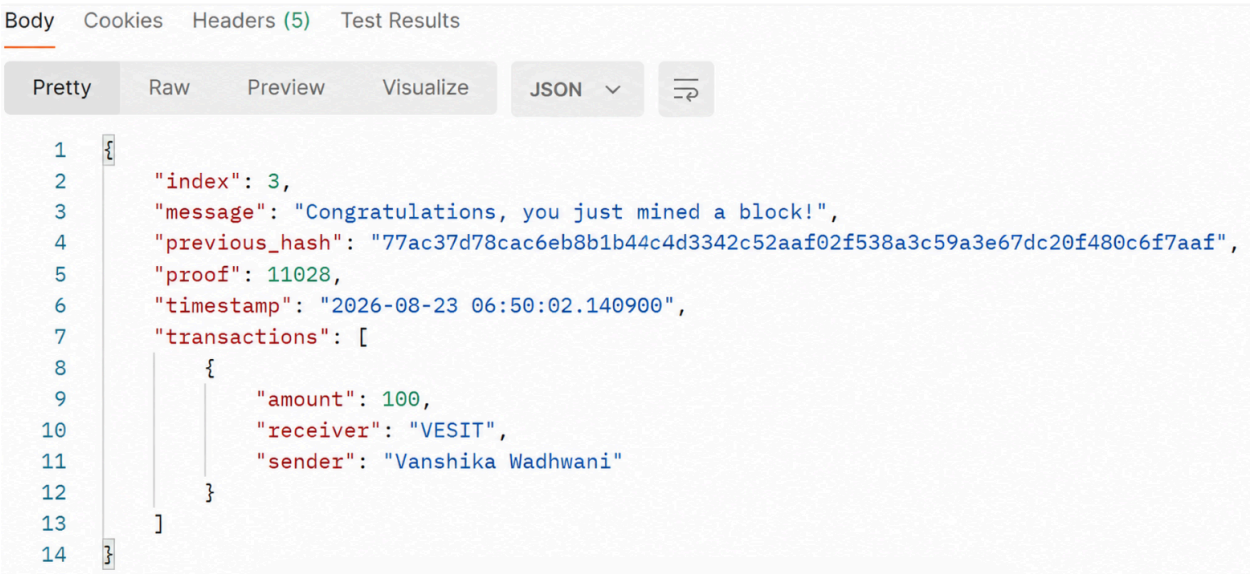
public_url = ngrok.connect(5000)

print("Flask
URL:")
print(public_url)
```

The generated URL was then used in Postman.

API 1 — Mine Block

Method: GET



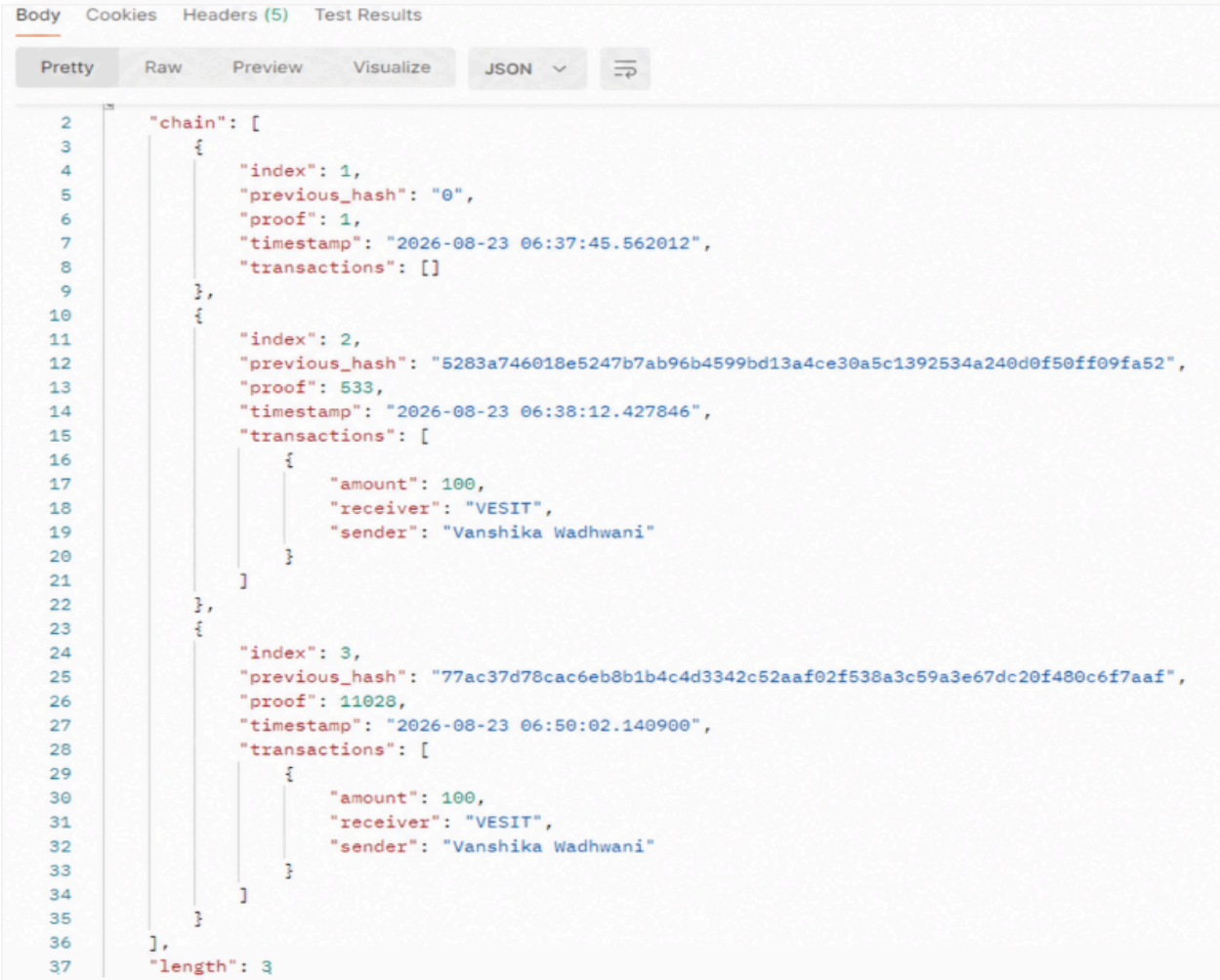
API 2 — Check Validity

Method:GET



API 3 — Get Blockchain

Method:GET



Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.